

Szeptemberi beszámoló

Dinamikus szobanövény-monitorozó rendszer hardveres és AI technológiával

A szeptemberi céljaim a szakdolgozatom keretében készülő projektem keretében a következők voltak:

- Szakirodalom gyűjtése
- Követelmények meghatározása
- Vercel és MongoDB környezet kialakítása
- Perenual API tesztelése
- IoT eszközök beszerzése

Első körben a hardveres eszközök beszerzésével kezdtem, mivel azoknak a beszerzési ideje, kiszállítása több időt vett igénybe. Az eszközök kiválasztásánál a fő szempont az volt, hogy egy stabil háttérre legyen mindegyiknek, jól dokumentálva, illetve, hogy minden kompatibilis legyen egymással. Számos szakirodalom, melyeket el is helyeztem a docs mappában, az ESP32 mikrokontrollert idézte egy "kötelező" alapnak, mely az agya is egyben a projektünknek. Ez irányítja az egyes szenzorokat, és ez küld wi-fi-n keresztül adatokat a szerver felé. Másik gyakori szenzor a DHT-22 volt, mely a hőmérsékletet és a páratartalmat méri. Minden szenzorból igyekeztem a modulos verziót választani, melyet a BB-102-es próbapanelre egyszerűen rá tudok kötni forrasztás nélkül. Van tapasztalatom a forrasztásokkal, viszont ennél a projektnél el szeretném kerülni, törekszek a legegyszerűbb megoldásokra, mert így kevesebb hiba fordulhat elő. A megvásárolt alkatrészek, szenzorok pontos listája a következő:

- ESP-WROOM-32-ESP32S (mikrokontroller)
- DHT22-M (hőmérséklet, páratartalom)
- GY-30 - BH1750FVI (környezeti fény)
- SOILCAP-V20 (talajnedvesség)
- BB-102 (630/200) (próbapanel)
- RC-10-20/MF (kábelek)
- RC-10-20/FF (kábelek)
- RC-10-20/MM (kábelek)
- CCP-MUSB2-AMBM-1M (micro usb kábel)

Egyelőre még nem teszteltem a beszerzett eszközöket, erre a projekt későbbi fázisában kerül sor.

A következő alapkőve a projektnek a tényleges Vercel, MongoDB és Next.js környezet kialakítása volt. A Next.js projekt inicializálásával kezdtem, melyet a dokumentáció segítségével a VS Code termináljában előre definiált parancsokkal végeztem el. Ezek előtt természetesen klónoztam lokálisan az előzetesen létrehozott Github repository-t, ezen belül a web mappában hoztam létre a projektet. Az ajánlott beállításokat választottam a telepítő kérdéseknél. Ez létrehozott egy alap fájl- és mappastruktúrát, amely megfelel a modern webfejlesztési irányelveknek, tökéletesen helyet adva a React-nek is. Volt egy probléma, amibe az inicializálás során ütköztem, a már létező web mappán belül egy további web mappán belül jött létre a projekt, melyet a létrehozott fájlok és mappák áthelyezésével orvosoltam.

A Next.js után következett a MongoDB Atlas környezet kialakítása. Már volt egy meglévő fiókom, ezen belül hoztam létre egy új projektet, ezen belül egy új "Cluster"-t, mely gyakorlatilag a MongoDB megfelelője az adatbázisnak, ez fog szolgálni az IoT eszközökről beérkező adatok tárolására. A clusterhez jelenleg az én felhasználói fiókom van hozzárendelve, viszont a Vercel akadálymentes működése miatt bármilyen IP-címről hozzáférhetővé tettem. Ezt a Database & Network Access menüpontra belül tettem lehetővé, a 0.0.0.0/0 IP-cím hozzáadásával. Ehhez az

adatbázishoz tartozik egy felhasználó, melyet fentebb említettem, illetve egy jelszó is. Ennek birtokában automatikusan létrehozott a MongoDB rendszere egy linket, amely a webes alkalmazást köti össze az adatbázissal. Mivel itt komoly biztonsági kockázatot jelenthet a jelszó a linkben, ezt a linket egy környezeti változóban tároltam el a Vercel-en belül.

A Vercel rendszerében úgyszintén volt már fiókom, így itt csak egyszerűen importáltam a Github repository-mat, így automatikusan össze tudta kapcsolni és létrehozni a projektet. A létrehozás során lehetett meghatározni az env változót is, melyen belül biztonságosan van tárolva az adatbázis hozzáférés. A legelső deploy során egy apró hibába ütköztem, amely a validator.ts fájlban jelentkezett. Erre a Vercel-en kívül a VS Code beépített kód ellenőrző alkalmazossága is felhívta a figyelmemet, ugyanis a `“./src/app/page”`, illetve a `“./src/app/layout”` kódrészlet hibásan jött létre a Next.js projekt inicializálásakor. A dupla web mappa miatt kétszer szerepelt a `“./”` rész, így több mappán akart átugrani a kód, illetve a fájl elnevezés eredetileg `“page.js”` volt, viszont a projekt létrehozásakor TypeScript nyelvet választottam, így eltávolítottam a kiterjesztést. A Next.js nagyszerűen kezeli a fájlneveket, automatikusan beazonosítja a megfelelő fájltypust kiterjesztés nélkül is. Miután orvosoltam ezt a problémát, a Vercel felületén egyből lefutott a deploy, így létre is jött a honlap élesített verziója, mely elérhető ezen a linken:

<https://plant-planner-iot-beta.vercel.app/>

Ezzel a projekt váza létre is jött, így ezt követően elkezdhetem a tényleges honlap kialakítását. A Vercel intelligens kialakítása miatt minden Github commit után automatikusan lefut a deploy is, így mindig naprakész verziót láthatunk az éles honlapon.

A következő problémába a Perenual API tesztelése során ütköztem. Ugyan a projekt céljainak megfelelő adatokat ad ki, sajnos mélyebb kutatás után rá kellett jönnöm, hogy az ingyenes plan csak az 1-3000-ig terjedő növények adatait adja ki, és sajnos több szobanövény, többek között a Monstera Deliciosa, mely a fő tesztelni kívánt növény a fejlesztésem során, nincs benne ebben a 3000-es intervallumban. Sajnos így ezt a megoldást nem tudom célirányosan, kényelmesen felhasználni.

További kutatás után rátaláltam egy alternatív, ingyenes megoldásra, amely stabilan tartalmazza a legtöbb szobanövényt, illetve pontosabb, a szituációhoz jobban illő adatokat szolgáltat. A [kaggle.com](https://www.kaggle.com) egy elterjedt, javarészt ingyenes dataset-eket tartalmazó felület. Rövidebb böngészés után több alkalmas dataset-et találtam, de a végleges választásom erre esett:

<https://www.kaggle.com/datasets/souvikrana17/indoor-plant-health-and-growth-dataset>

Ez a dataset 1000 szobanövény adatait tartalmazza, specializálva az adott növény igényeire, növekedését elősegítő tényezőkre. Ezzel látványosabb növény adatlapokat tudok létrehozni, illetve az IoT szenzoroktól érkező adatokat is pontosabban tudom felhasználni, tippeket és riasztásokat kiadni a felhasználó részére.

Ezt a dataset-et integrálnám a MongoDB adatbázisomba, és innen történne az adatok lekérdezése, ezzel megteremtve a saját API-mat. Azért tetszik jobban ez a megoldás, mert így nem kell egy külső API-ra támaszkodni, amelynek megvannak a saját, ingyenes korlátai, illetve így garantált a stabil, ingyenes működés, nem befolyásolnak külső, rajtunk kívül álló tényezők (pl. Perenual API ideiglenes leállása, napi limit elérése).

Ennek az integrálását az eredeti terv szerint tervezem ütemezni, a Perenual API-t lecserélve.

A projekt fejlesztését tovább folytatom, a fejleményekről továbbra is részletesen beszámolok a következő hónapokban.

Köszönöm a figyelmet!